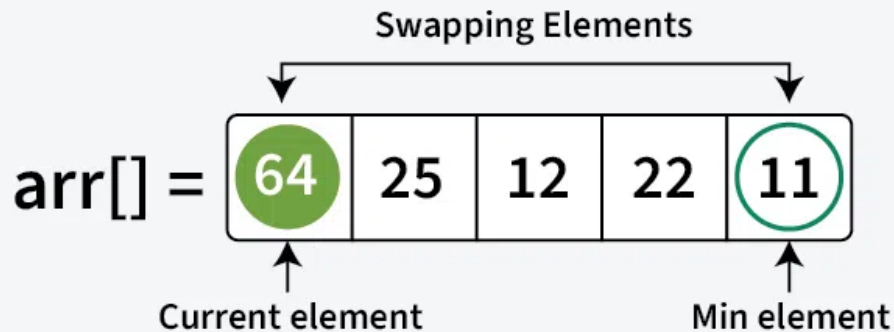


01
Step

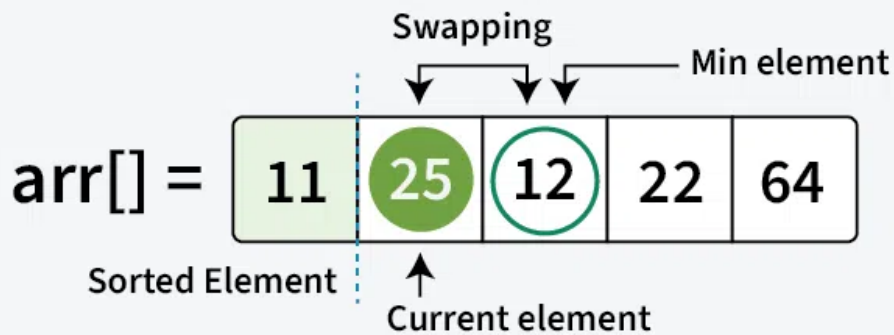
Start from the first element at index 0, find the smallest element in the rest of the array which is unsorted, and swap (11) with current element(64).



Selection Sort Algorithm

02
Step

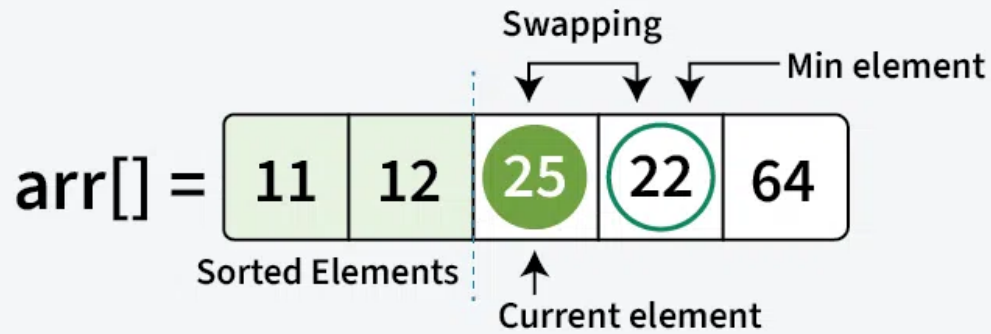
Move to the next element at index 1 (25). Find the smallest in unsorted subarray, and swap (12) with current element (25).



Selection Sort Algorithm

03
Step

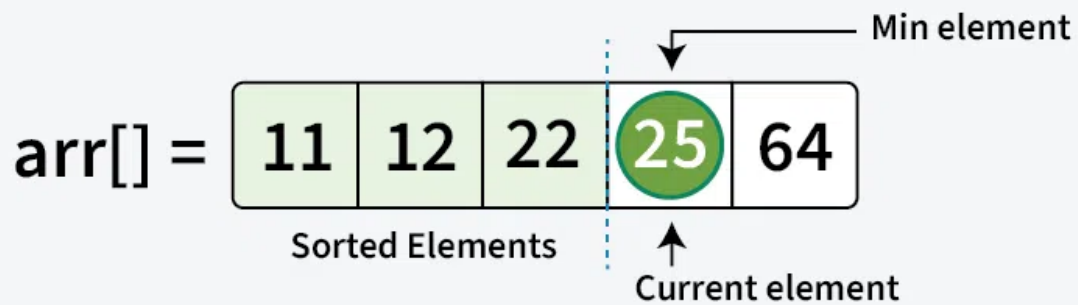
Move to element at index 2 (25). Find the minimum element from unsorted subarray, Swap (22) with current element (25).



Selection Sort Algorithm

04
Step

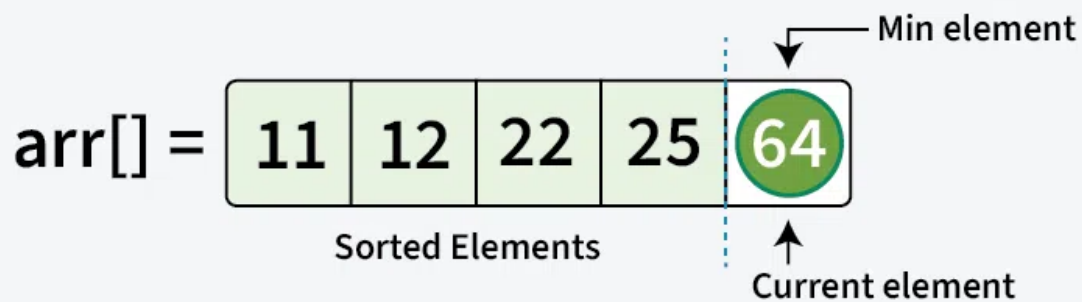
Move to element at index 3 (25), find the minimum from unsorted subarray and swap (25) with current element (25).



Selection Sort Algorithm

05
Step

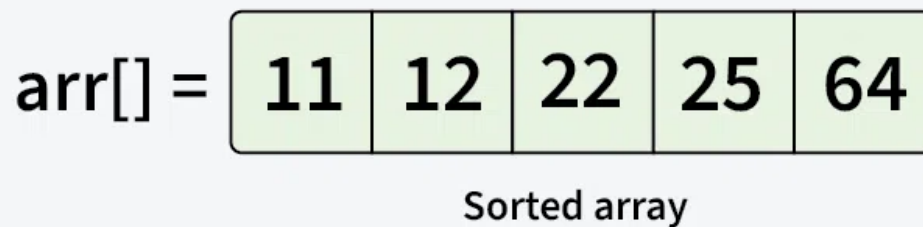
Move to element at index 4 (64), find the minimum from unsorted subarray and swap (64) with current element (64).



Selection Sort Algorithm

06
Step

We get the sorted array at the end.



Selection Sort Algorithm

การเรียงลำดับแบบเลือก

อัปเดตล่าสุด : 8 ธันวาคม 2025

การเรียงลำดับแบบเลือก (Selection Sort) เป็นอัลกอริทึมการเรียงลำดับแบบเปรียบเทียบ โดยจะเรียงลำดับด้วยการเลือกองค์ประกอบ ที่เล็กที่สุด (หรือใหญ่ที่สุด) จากส่วนที่ยังไม่ได้เรียงลำดับซ้ำๆ และสลับตำแหน่งกับองค์ประกอบแรกที่ยังไม่ได้เรียงลำดับ

1. หาสององค์ประกอบที่เล็กที่สุดแล้วสลับตำแหน่งกับองค์ประกอบแรก วิธีนี้จะทำให้สององค์ประกอบที่เล็กที่สุดกลับไปอยู่ในตำแหน่งที่ถูกต้อง
2. จากนั้นให้หาค่าที่เล็กที่สุดในบรรดาค่าที่เหลืออยู่ (หรือค่าที่เล็กเป็นอันดับสอง) แล้วสลับกับค่าที่สอง
3. เราทำเช่นนี้ต่อไปเรื่อยๆ จนกว่าองค์ประกอบทั้งหมดจะถูกจัดวางในตำแหน่งที่ถูกต้อง

// C++ program to implement Selection Sort

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
void selectionSort(vector<int> &arr) {  
    int n = arr.size();
```

```
    for (int i = 0; i < n - 1; ++i) {
```

```
        // Assume the current position holds  
        // the minimum element  
        int min_idx = i;
```

```
        // Iterate through the unsorted portion  
        // to find the actual minimum  
        for (int j = i + 1; j < n; ++j) {  
            if (arr[j] < arr[min_idx]) {
```

```
                // Update min_idx if a smaller  
                // element is found  
                min_idx = j;
```

```
            }  
        }
```

```
        // Move minimum element to its  
        // correct position  
        swap(arr[i], arr[min_idx]);
```

```
    }  
}
```

```
void printArray(vector<int> &arr) {
```

```

    for (int &val : arr) {
        cout << val << " ";
    }
    cout << endl;
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};

    cout << "Original array: ";
    printArray(arr);

    selectionSort(arr);

    cout << "Sorted array: ";
    printArray(arr);

    return 0;
}

```

เอาต์พุต

เวกเตอร์ดั้งเดิม: 64 25 12 22 11

เวกเตอร์ที่เรียงลำดับแล้ว: 11 12 22 25 64

การวิเคราะห์ความซับซ้อนของการเรียงลำดับแบบเลือก

ความซับซ้อนของเวลา: $O(n^2)$ เนื่องจากมีลูปซ้อนกันสองชั้น:

- การเลือกองค์ประกอบจากอาร์เรย์ทีละรายการโดยใช้ลูปหนึ่งครั้ง = $O(n)$
- วนลูปอีกครั้งเพื่อเปรียบเทียบองค์ประกอบนั้นกับองค์ประกอบอื่นๆ ทุกตัวในอาร์เรย์ = $O(n)$
- ดังนั้นความซับซ้อนโดยรวม = $O(n) * O(n) = O(n * n) = O(n^2)$

พื้นที่เสริม: $O(1)$ เนื่องจากหน่วยความจำเสริมที่ใช้มีเพียงสำหรับตัวแปรชั่วคราวเท่านั้น

ข้อดีของการเรียงลำดับแบบเลือก (Selection Sort)

- เข้าใจง่ายและนำไปใช้ได้จริง ทำให้เหมาะอย่างยิ่งสำหรับการสอนแนวคิดพื้นฐานเกี่ยวกับการจัดเรียง
- ต้องการพื้นที่หน่วยความจำเพิ่มเติมเพียงค่าคงที่ $O(1)$ เท่านั้น
- อัลกอริทึมนี้ใช้จำนวนการสลับข้อมูล (หรือการเขียนข้อมูลลงหน่วยความจำ) น้อยกว่าอัลกอริทึมมาตรฐานอื่นๆ หลายตัว มีเพียง อัลกอริทึม [Cycle Sort](#) เท่านั้น ที่ใช้จำนวนการเขียนข้อมูลลงหน่วยความจำมากกว่า ดังนั้นจึงเป็นตัวเลือกอัลกอริทึมที่เหมาะสมเมื่อการเขียนข้อมูลลงหน่วยความจำมีค่าใช้จ่ายสูง

ข้อเสียของการเรียงลำดับแบบเลือก (Selection Sort)

- อัลกอริทึม Selection Sort มีความซับซ้อนเชิงเวลาอยู่ที่ $O(n^2)$ ทำให้ช้ากว่าอัลกอริทึมอย่าง [Quick Sort](#) หรือ [Merge Sort](#)

- ไม่รักษาระดับความสัมพันธ์ขององค์ประกอบที่เท่ากัน ซึ่งหมายความว่ามันไม่เสถียร

การประยุกต์ใช้การเรียงลำดับแบบเลือก

- เหมาะอย่างยิ่งสำหรับการสอนกลไกการเรียงลำดับพื้นฐานและการออกแบบอัลกอริทึม
- เหมาะสำหรับรายการขนาดเล็กที่ไม่คุ้มค่ากับค่าใช้จ่ายของอัลกอริทึมที่ซับซ้อนกว่า และการเขียนข้อมูลลงหน่วยความจำมีค่าใช้จ่ายสูง เนื่องจากใช้การเขียนข้อมูลลงหน่วยความจำน้อยกว่าเมื่อเทียบกับอัลกอริทึมการเรียงลำดับมาตรฐานอื่นๆ
- อัลกอริทึม Heap Sort พัฒนามาจาก Selection Sort

คำถามที่ 1: การเรียงลำดับแบบ Selection Sort เป็นอัลกอริทึมการเรียงลำดับที่มีเสถียรภาพหรือไม่?

คำตอบ: ไม่ การเรียงลำดับแบบเลือก (Selection Sort) ไม่ใช่การเรียงลำดับแบบเสถียรเนื่องจากอาจเปลี่ยนแปลงลำดับสัมพัทธ์ขององค์ประกอบที่เท่ากันได้

คำถามที่ 2: ความซับซ้อนเชิงเวลาของการเรียงลำดับแบบเลือก (Selection Sort) คือเท่าไร?

คำตอบ: การเรียงลำดับแบบ Selection Sort มีความซับซ้อนเชิงเวลาอยู่ที่ $O(n^2)$ ในกรณีที่แย่ที่สุด กรณีเฉลี่ย และกรณีที่แย่ที่สุด

คำถามที่ 3: การเรียงลำดับแบบ Selection Sort ต้องการหน่วยความจำเพิ่มเติมหรือไม่?

คำตอบ: ไม่ Selection Sort เป็นอัลกอริทึมการเรียงลำดับแบบ in-place และต้องการพื้นที่ที่เพิ่มเติมเพียง $O(1)$ เท่านั้น

คำถามที่ 4: เมื่อใดจึงเหมาะสมที่สุดที่จะใช้การเรียงลำดับแบบเลือก (Selection Sort)?

คำตอบ: การเรียงลำดับแบบ Selection Sort เหมาะที่สุดสำหรับชุดข้อมูลขนาดเล็ก เพื่อการศึกษา หรือเมื่อต้องการลดการใช้หน่วยความจำให้น้อยที่สุด

คำถามที่ 5: การเรียงลำดับแบบ Selection Sort แตกต่างจากการเรียงลำดับแบบ Bubble Sort อย่างไร?

คำตอบ: การเรียงลำดับแบบ Selection Sort จะเลือกองค์ประกอบที่เล็กที่สุดและวางไว้ในตำแหน่งที่ถูกต้องโดยใช้การสลับตำแหน่งน้อยกว่า ในขณะที่การเรียงลำดับแบบ Bubble Sort จะสลับตำแหน่งองค์ประกอบที่อยู่ติดกันซ้ำๆ เพื่อเรียงลำดับอาร์เรย์

Selection Sort Algorithm

Selection sort is a [sorting algorithm](#) that selects the smallest element from an unsorted list in each iteration and places that element at the beginning of the unsorted list.

Working of Selection Sort

1. Set the first element



as `minimum`.

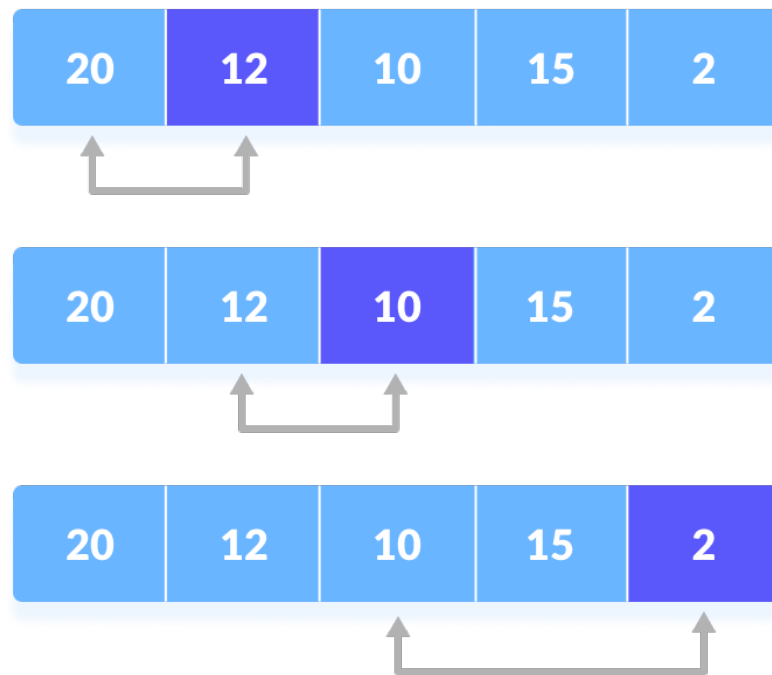
Select

first element as minimum

2. Compare `minimum` with the second element. If the second element is smaller than `minimum`, assign the second element as `minimum`.

Compare `minimum` with the third element. Again, if the third element is smaller, then assign `minimum` to the third element otherwise do nothing.

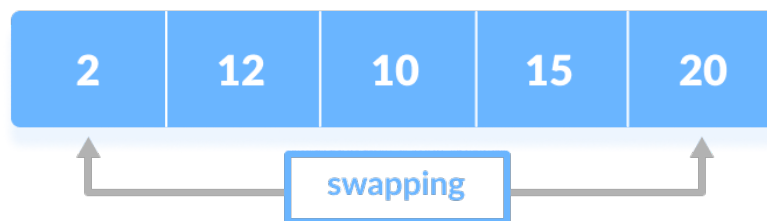
The process goes on until the last



element.

Compare minimum with the remaining elements

3. After each iteration, `minimum` is placed in the front of the unsorted



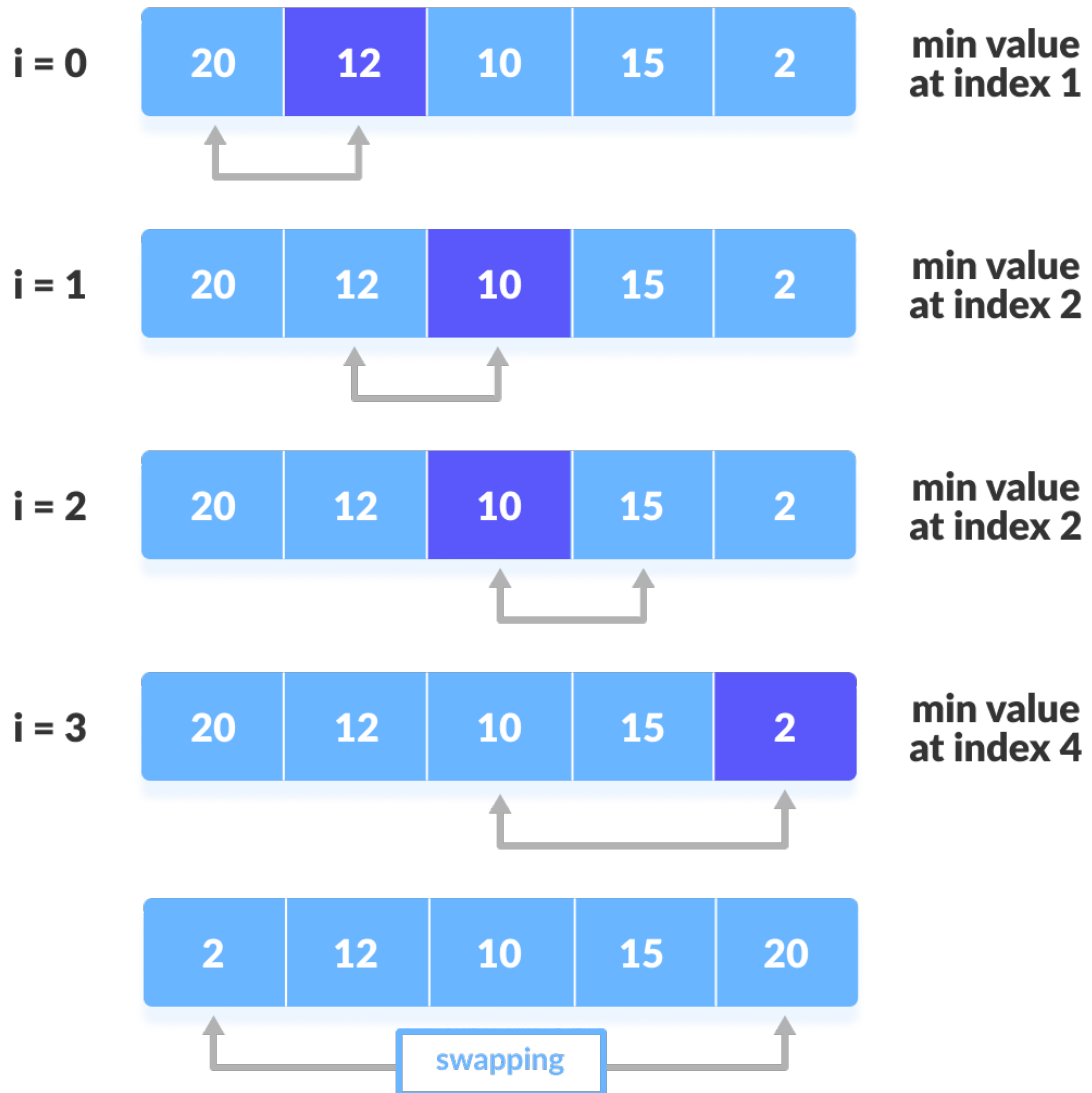
list.

Swap the first

with minimum

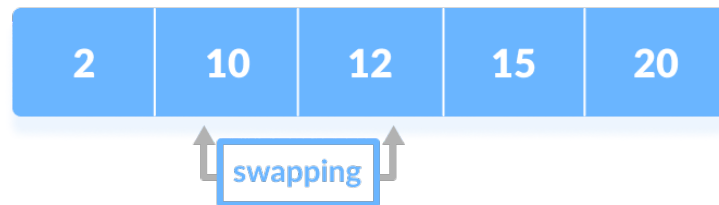
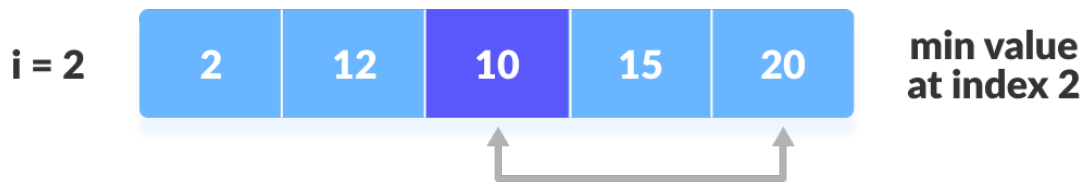
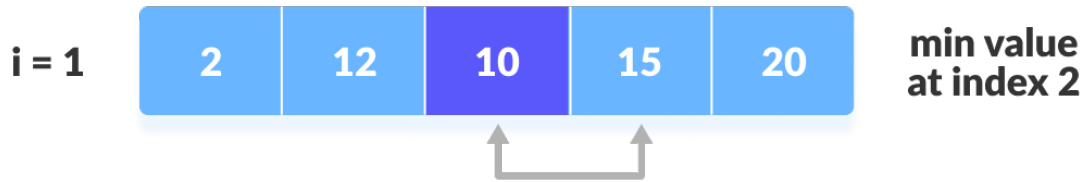
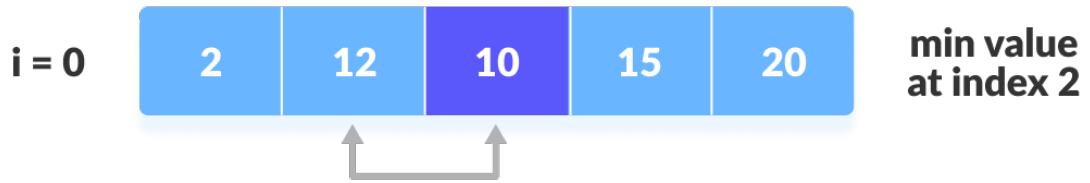
4. For each iteration, indexing starts from the first unsorted element. Step 1 to 3 are repeated until all the elements are placed at their correct positions.

step = 0



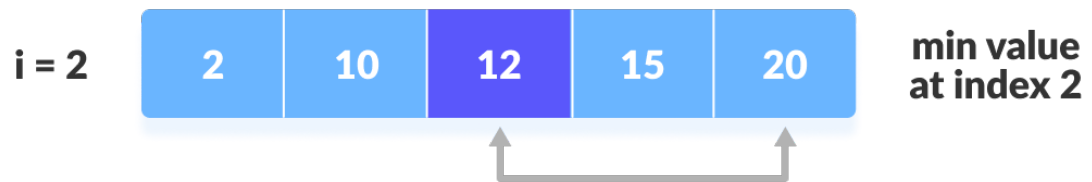
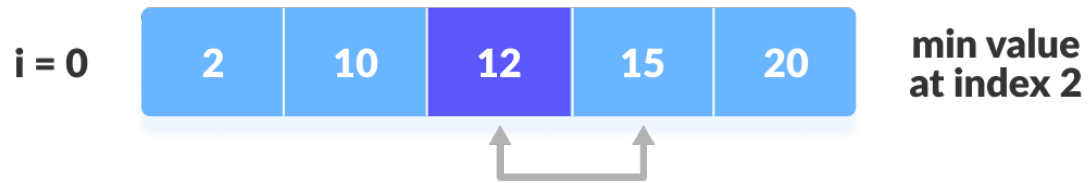
The first
iteration

step = 1



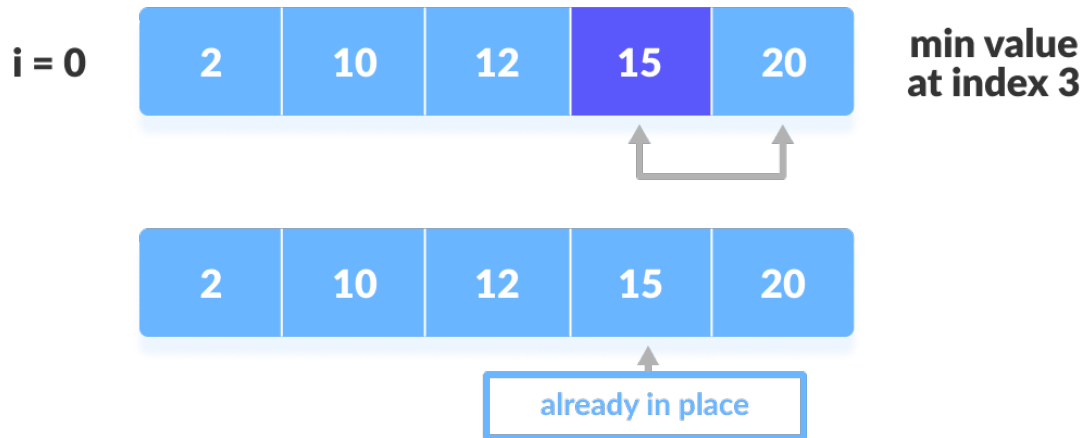
The second
iteration

step = 2



The third
iteration

step = 3



The fourth iteration

Selection Sort Algorithm

```
selectionSort(array, size)
  for i from 0 to size - 1 do
    set i as the index of the current minimum
    for j from i + 1 to size - 1 do
      if array[j] < array[current minimum]
        set j as the new current minimum index
    if current minimum is not i
      swap array[i] with array[current minimum]
  end selectionSort
```

Selection Sort Code in Python, Java, and C/C++

Python

Java

C

C++

```
# Selection sort in Python

def selectionSort(array, size):

    for step in range(size):
        min_idx = step

        for i in range(step + 1, size):

            # to sort in descending order, change > to < in this line
            # select the minimum element in each loop
            if array[i] < array[min_idx]:
                min_idx = i

        # put min at the correct position
        (array[step], array[min_idx]) = (array[min_idx], array[step])

data = [-2, 45, 0, 11, -9]
size = len(data)
selectionSort(data, size)
print('Sorted Array in Ascending Order:')
print(data)
```

Selection Sort Complexity

Time Complexity

Best	$O(n^2)$
Worst	$O(n^2)$
Average	$O(n^2)$
Space Complexity	$O(1)$
Stability	No

Cycle	Number of Comparison
1st	$(n-1)$
2nd	$(n-2)$
3rd	$(n-3)$
...	...
last	1

Number of comparisons: $(n - 1) + (n - 2) + (n - 3) + \dots + 1 = n(n - 1) / 2$

$\approx$ nearly equals to n^2 .

Complexity = $O(n^2)$

Also, we can analyze the complexity by simply observing the number of loops.

There are 2 loops so the complexity is $n \times n = n^2$.

Time Complexities:

- **Worst Case Complexity:** $O(n^2)$

If we want to sort in ascending order and the array is in descending order then, the worst case occurs.

- **Best Case Complexity:** $O(n)$

It occurs when the array is already sorted

- **Average Case Complexity:** $O(n^2)$

It occurs when the elements of the array are in jumbled order (neither ascending nor descending).

The time complexity of the selection sort is the same in all cases. At every step, you have to find the minimum element and put it in the right place. The minimum element is not known until the end of the array is not reached.

Space Complexity:

Space complexity is $O(1)$ because an extra variable `min_idx` is used.

Selection Sort Applications

The selection sort is used when

- a small list is to be sorted
 - cost of swapping does not matter
 - checking of all the elements is compulsory
 - cost of writing to a memory matters like in flash memory (number of writes/swaps is $O(n)$ as compared to $O(n^2)$ of bubble sort)
-

อัลกอริทึม Selection Sort คืออะไร?

เมื่อพูดถึงการจัดเรียงข้อมูล (Sorting), ความสามารถในการเรียงลำดับองค์ประกอบต่างๆ ตามเกณฑ์ที่กำหนดไว้เป็นการดำเนินการพื้นฐานที่พบในหลายระบบคอมพิวเตอร์และแอปพลิเคชันต่างๆ หากเราจะกล่าวถึง "Selection Sort" อัลกอริทึมหนึ่งที่ได้รับการกล่าวขานอย่างกว้างขวาง ทั้งในห้องเรียนและในตำราเทคโนโลยีสารสนเทศ อัลกอริทึมนี้มีความเรียบง่ายแต่ทรงประสิทธิภาพในกรณีที่เหมาะสม จุดเด่นของมันคือความสามารถในการค้นหาและเลือก "ขั้นต่ำ" (Min) หรือ "ขั้นสูงสุด" (Max) จากลิสต์ข้อมูลแล้วสลับข้อมูลนั้นไปยังตำแหน่งที่ถูกต้อง นี่คือการวนการที่เกิดขึ้นในแต่ละรอบจนข้อมูลทั้งหมดถูกจัดเรียงอย่างเรียบร้อย

ตัวอย่างการใช้งาน Selection Sort

ในภาษา C# การเขียนโค้ดสำหรับ Selection Sort สามารถแสดงได้ดังนี้:

```
public void SelectionSort(int[] arr)
{
    int n = arr.Length;

    for (int i = 0; i < n - 1; i++)
    {
        // ค้นหาตำแหน่งขั้นต่ำในส่วนที่ยังไม่ได้เรียงของ array
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }

        // สลับตำแหน่งขั้นต่ำที่พบกับตำแหน่งแรกของส่วนที่ยังไม่ได้เรียง
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}
```

Usecase ในโลกจริง

Selection Sort มีความสะดวกในกรณีที่เรากำลังเรียงลำดับข้อมูลที่มีจำนวนน้อยถึงปานกลาง เช่น การจัดเรียงรายชื่อนักเรียนในชั้นเรียน, การจัดเรียงสินค้าตามรหัสในระบบคลังสินค้าขนาดเล็ก หรือการใช้ในโปรแกรมที่ไม่จำเป็นต้องคำนึงถึงประสิทธิภาพการทำงานระดับสูงสุด

Complexity ของ Selection Sort

ในแง่ของเวลาที่ใช้ (Time Complexity), Selection Sort มีความซับซ้อนแบบ $O(n^2)$ ซึ่งโดยทั่วไปไม่ค่อยเหมาะกับชุดข้อมูลขนาดใหญ่ เนื่องจากประสิทธิภาพจะลดลงอย่างมีนัยสำคัญเมื่อข้อมูลมีขนาดเพิ่มขึ้น

ข้อดีข้อเสียของ Selection Sort

ข้อดี:

- ความเรียบง่ายของโค้ดและความเข้าใจง่าย
- ไม่จำเป็นต้องใช้หน่วยความจำเพิ่มเติม (In-place Sorting)
- การสลับทำได้น้อยครั้งเมื่อเทียบกับบางอัลกอริทึมอื่น ซึ่งดีสำหรับกรณีที่การสลับมีค่าใช้จ่ายสูง

ข้อเสีย:

- มีประสิทธิภาพไม่ดีเมื่อเทียบกับอัลกอริทึมอื่นๆ เช่น Quick Sort, Merge Sort ในชุดข้อมูลขนาดใหญ่
- อัลกอริทึมไม่มีการถูกปรับปรุงหรือเปลี่ยนแปลงเมื่อข้อมูลเรียงลำดับมาก่อนแล้ว (Non-Adaptive)

สรุป

Selection Sort เป็นหนึ่งในอุปกรณ์พื้นฐานที่นักพัฒนาซอฟต์แวร์ควรมีในเวทีของการเขียนโปรแกรม ไม่ว่าจะเป็นการใช้งานจริง หรือการเรียนรู้หลักการจัดเรียงข้อมูล ที่ **Expert-Programming-Tutor** หรือ EPT, เราสนับสนุนและขอเชิญชวนให้นักเรียนทุกท่านทำความเข้าใจอัลกอริทึมเหล่านี้ในระดับลึก และใช้เป็นพื้นฐานอันมั่นคงสำหรับการเติบโตและพัฒนาเพิ่มเติมในโลกของการเขียนโปรแกรม สนใจดูรายละเอียดคอร์สเรียนรู้พิเศษได้ที่ EPT ที่จะนำคุณไปพบกับโลกใหม่ที่เต็มไปด้วยโอกาสและความท้าทาย!